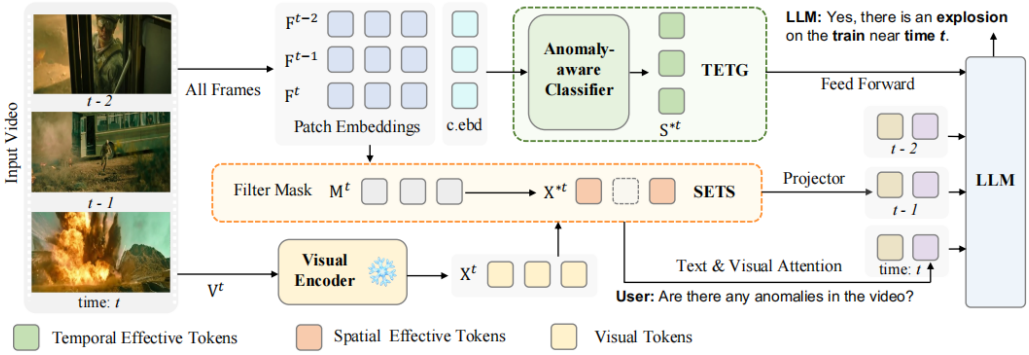

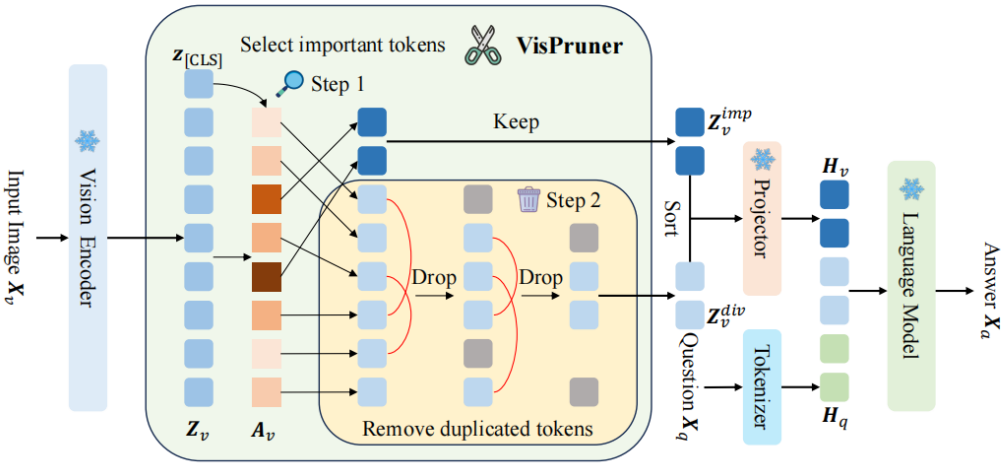

基于两张论文示意图的论文解读：VA-GPT 与 VisPruner

一、整体概述

这两篇论文都围绕同一个核心问题展开：**视觉大模型中的视觉 token 存在大量冗余，若把所有 token 不加区分地送入大语言模型，不仅计算代价高，还会稀释真正关键的视觉信息。**不过，两篇工作的研究目标并不相同：

- **VA-GPT** 关注的是**视频异常理解**，目标是让模型更准确地回答“视频里有没有异常、异常发生在什么地方、发生在什么时间”。
- **VisPruner** 关注的是**通用视觉语言模型的高效推理**，目标是在尽量不损失性能的前提下，大幅减少视觉 token，从而降低 FLOPs、显存占用和推理时延。

因此，可以把两篇论文概括为：

> **VA-GPT 更强调“看得准”**，解决的是异常事件在空间和时间上都很稀疏的问题；
> **VisPruner 更强调“跑得快”**，解决的是视觉 token 太多导致推理成本过高的问题。

二、第一篇论文：VA-GPT

1. 论文要解决什么问题

VA-GPT 的出发点是：虽然现有视频理解多模态大模型已经能处理一般视频，但在**异常视频理解**上仍然表现不足。根本原因在于，异常事件通常具有两个典型特点：

1. **空间稀疏**：异常只发生在画面中的局部区域；
2. **时间稀疏**：异常只发生在视频中的少数片段。

如果模型仍像普通视频理解那样，把所有帧、所有 patch token 都等价处理，就会把大量与异常无关的背景信息一起送入 LLM，导致真正重要的异常信号被淹没。

因此，VA-GPT 的核心思想是：**不要平均对待所有视觉 token，而是从空间和时间两个维度，挑选真正与异常相关的有效 token，与 LLM 对齐。**

2. 结合第一张图理解 VA-GPT 的结构

第一张图展示了 VA-GPT 的完整框架。整条流程可以分成两条主线：

（1）空间主线：SETS

图中下方橙色虚线框部分对应 **SETS（Spatial Effective Token Selection，空间有效 token 选择）**。

它的处理逻辑是：

- 输入视频后，模型先从当前帧和相邻帧中提取 patch embeddings；
- 比较相邻帧之间每个 patch 的特征差异；
- 差异大的 patch 往往意味着该区域发生了明显变化，更可能与异常有关；
- 因此，模型根据这种差异构造一个 **Filter Mask**，把不重要的视觉 token 过滤掉；
- 最终只保留少量更关键的 **Spatial Effective Tokens**，再通过 projector 映射到 LLM 的语义空间中。

从图上可以直观看出，SETS 的作用类似于“把镜头聚焦到真正发生异常的位置”，避免模型被背景区域干扰。

（2）时间主线：TETG

图中右上方绿色虚线框部分对应 **TETG（Temporal Effective Token Generation，时间有效 token 生成）**。

它的逻辑是：

- 模型先从所有帧中提取 class embedding；
- 再利用一个轻量级的 **Anomaly-aware Classifier** 对每一帧进行评分，判断该帧是否可能包含异常；
- 根据这些分数，找到异常事件大致开始和结束的时间段；
- 然后把这个时间先验信息转换成文本形式的 token，即图中的 **Temporal Effective Tokens**；
- 最后将这些 token 作为额外输入送入 LLM。

这意味着，VA-GPT 不只是“让模型自己看”，而是显式地告诉 LLM：**异常大概发生在什么时间范围内。**这样模型在回答“异常是否存在、异常什么时候发生”这类问题时会更稳定、更准确。

（3）两条主线如何协同

如果说 SETS 解决的是“**看哪里**”的问题，那么 TETG 解决的就是“**看什么时候**”的问题。二者结合后，LLM 接收到的就不再是冗余、平均化的视觉输入，而是：

- 一部分来自空间上的异常关键区域；
- 一部分来自时间上的异常先验信息。

因此，图中最终的 LLM 才能回答类似：**“是的，在时间 t 附近，列车处发生了爆炸。”**

3. 方法细节

（1）SETS 的本质

SETS 的核心不是简单做图像裁剪，而是通过**相邻帧差异**来筛选更有价值的 patch token。论文中使用 DINOv2 提取相邻帧的 patch embedding，再计算 patch 级别的距离，并保留变化最大的 top-K 区域。这种做法的直觉非常合理：异常通常伴随局部显著变化，例如撞击、爆炸、打斗、火焰蔓延等。

（2）TETG 的本质

TETG 的关键创新点在于：它不是把时间信息继续编码成视觉特征，而是**直接转成自然语言模板 token**，送入 LLM 的文本空间。也就是说，作者把“异常的时间定位”从视觉侧抽出来，变成 LLM 更容易利用的语言先验。

这种设计说明作者并不只是想减少 token，而是在思考：**怎样让 LLM 更自然地利用异常时间信息。**

（3）训练策略

VA-GPT 采用两阶段训练：

- **Stage One**：先用异常视频数据进行 instruction tuning，让模型先具备异常理解能力；
- **Stage Two**：再用 SETS 筛选出的空间有效 token 继续对齐微调，强化视觉异常区域与文本语义之间的映射关系。

此外，作者还构建了专门的异常视频 instruction-following 数据集，使模型不仅能做检测，还能做问答、描述和时间定位。

4. 实验结果说明了什么

实验表明，VA-GPT 在两个层面都取得了提升：

（1）主结果

在 **UCF-Crime**（域内）和 **XD-Violence**（跨域）两个基准上，VA-GPT 都取得了最优表现：

- UCF-Crime：Total Acc. = **30.69%**，Temporal Acc. = **35.00%**；
- XD-Violence：Total Acc. = **26.20%**，Temporal Acc. = **31.02%**。

这说明它不仅在训练域内有效，而且在跨域异常视频上也具备更强的泛化能力。

（2）消融实验

消融实验显示：

- 单独使用 SETS 或单独使用 TETG 都有提升；
- 两者同时使用效果最好；
- 经过第二阶段微调后，模型性能进一步提高。

这说明论文中的两个模块并不是彼此替代，而是互补关系：一个负责空间筛选，一个负责时间增强。

5. 这篇论文的价值与局限

价值

VA-GPT 的价值在于，它不是单纯追求更大的模型规模，而是提出了一个**面向异常视频理解的 token 对齐机制**。它把异常理解问题拆成“空间关键区域选择”和“时间关键片段提示”两个部分，这种建模方式非常贴合异常任务本身的特点。

可能的局限

不过，VA-GPT 也有一定局限：

- 它较强地依赖“异常通常伴随局部显著变化”这一假设；
- 对于变化不明显、但语义上异常的场景，效果可能未必同样突出；
- 它更像是为“异常视频理解”定制的方法，任务专属性较强，通用性不如通用加速框架。

三、第二篇论文：VisPruner

1. 论文要解决什么问题

VisPruner 关注的是另一个更基础的问题：**视觉语言模型中的视觉 token 太多，推理代价太高。**

例如，一张图像在 LLaVA-1.5 中会被编码成数百个视觉 token，高分辨率图像和视频更是会产生更长的 token 序列。此前很多工作尝试根据**文本-视觉注意力**来决定哪些 token 应该被保留，但作者发现：**这种做法并不可靠。**

2. 结合第二张图理解 VisPruner 的结构

第二张图展示了 VisPruner 的整体流程。和 VA-GPT 不同，它不是针对某一类特殊任务，而是一个通用的、训练免费（training-free）的视觉 token 剪枝模块。

整个流程分为两个步骤。

（1）Step 1：选择重要 token

图中上半部分的 **Step 1** 表示：

- 输入图像先经过 Vision Encoder，得到视觉 token；
- 然后利用视觉编码器内部的注意力，选择一小部分最重要的 token；
- 这些 token 被直接 **Keep** 下来。

注意，这里依赖的是**视觉编码器自身的视觉注意力**，而不是语言模型中的 text-visual attention。作者认为，视觉编码器更知道哪些 patch 本身具有丰富视觉信息，因此更适合做“重要 token”的判断。

（2）Step 2：移除重复 token，保留多样 token

如果只保留“最重要”的 token，模型很可能只盯着前景目标，忽略背景信息。因此图中下半部分的 **Step 2** 又做了一件事：

- 对剩余 token 计算相似度；
- 逐步移除那些彼此过于相似、信息重复的 token；
- 最后保留一部分**更有多样性**的 token。

这部分 token 不是最显著的，但它们能够补充背景、环境和上下文信息。最终保留下来的 token 由两部分组成：

- **important tokens**：保证关键信息不丢；
- **diverse tokens**：保证上下文覆盖更完整。

（3）最终输入 LLM

图中可以看到，这两类 token 会先经过 projector，再与文本问题一起输入 Language Model。也就是说，VisPruner 的目标不是改变大模型本身，而是在进入语言模型之前，先把视觉输入“压缩成更精华的版本”。

3. 为什么作者不相信 text-visual attention

VisPruner 论文最有价值的部分，不只是提出方法，还先解释了：**为什么已有方法不够好。**

作者给出了两个关键分析：

（1）Attention Shift

由于语言模型中的 rotary position embedding 存在位置偏置，当输入结构是 `[image, text]` 时，后面的文本 token 更容易关注离自己更近的视觉 token，也就是图像中位置更靠后的 patch。结果就是，模型会偏向保留某些固定位置的 token，而不一定是真正重要的视觉内容。

（2）Attention Dispersion

即使把位置偏置问题缓解之后，text-visual attention 仍然不够“尖锐”。换句话说，它在很多视觉 token 上分布得比较平均，不像视觉编码器中的 [CLS] attention 那样更容易区分“哪些 token 真正重要”。因此，如果直接拿它做剪枝依据，性能就容易下降。

这两点分析实际上构成了 VisPruner 的理论基础：**剪枝应该更多依赖视觉线索，而不是语言模型内部的文本-视觉注意力。**

4. 方法细节

（1）重要 token

VisPruner 首先根据视觉编码器中的 [CLS] attention（或等价视觉注意力）挑选重要 token。这样选出的 token 更偏向图像中的显著语义区域，能够优先保留真正有判别性的内容。

（2）多样 token

随后，它在剩余 token 里做相似度去重，留下不重复、覆盖更广的 token。这个设计非常关键，因为只保留“最重要”的 token 可能会让模型过度集中于前景，从而丢失背景信息；而 diverse token 恰好弥补了这一点。

5. 实验结果说明了什么

VisPruner 的实验结果非常强，尤其是在大幅压缩 token 数量时仍能保持性能。

（1）在 LLaVA-1.5-7B 上

论文中给出的结果显示：

- 保留 **128 个 token**（剪掉 77.8%）时，平均性能还能保持 **99.6%**；
- 保留 **64 个 token**（剪掉 88.9%）时，仍保持 **96.6%**；
- 保留 **32 个 token**（剪掉 94.4%）时，仍保持 **91.5%**。

这说明 VisPruner 在极高压缩率下仍能稳定保留关键信息。

（2）效率收益

论文摘要中指出，在 **LLaVA-1.5-7B** 上，VisPruner 可以在**无需额外训练**的情况下，将 FLOPs 降低 **91%**，将推理时延降低 **75%**。这表明它不仅“压得多”，而且“真的更快”。

（3）高分辨率和视频场景

作者还把 VisPruner 应用到 **LLaVA-NeXT** 和 **Video-LLaVA** 上。结果显示，在高分辨率图像和视频这种视觉 token 更长的场景里，VisPruner 依然优于 FastV、SparseVLM、VisionZip 等方法，说明它具有较强的通用性。

6. 这篇论文的价值与局限

价值

VisPruner 的最大价值在于它是一个**即插即用、无需训练、适用于多种 VLM 架构**的通用剪枝方法。它不是针对某一个下游任务单独优化，而是提供了一种更合理的视觉 token 压缩原则。

可能的局限

它的局限主要在于：

- 目标更偏向**高效推理**，而不是面向某个专门任务的极致性能提升；
- 尽管它尽量保留了信息，但在极端压缩下仍不可避免会有一定性能损失；
- 它解决的是“通用 token 冗余”问题，而不是像 VA-GPT 那样显式建模异常事件的时间定位。